



Python Fundamentos

Introducción

1



Un archivo constituye una colección de datos interrelacionados, los cuales se almacenan de manera permanente en dispositivos como discos duros, memorias flash, entre otros, y pueden ser considerados, en cierta medida, como una entidad singular.

La noción de archivo representa una abstracción del Sistema Operativo (S.O.), que de manera transparente al desarrollador, emplea los recursos del hardware para establecer una estructura lógica sobre ellos, simplificando así a los lenguajes de programación el acceso a interfaces de usuario que sean sencillas y adaptables.

El Sistema Operativo se encarga de generar la totalidad del sistema de archivos de la computadora: desde los directorios destinados a organizarlos hasta las tablas para ubicarlos en el disco, entre otras funciones.

Para acceder a un archivo, se debe utilizar la siguiente declaración:

```
archivo=open("archivo.txt",[modo_apertura])
```

Modos de
apertura

2



Los modos de apertura son parámetros utilizados para abrir ficheros con la función 'open()'. Nos permiten concretar el método de lectura y escritura, ya que habrá situaciones en las que queramos sólo leer un fichero, sobreescribirlo o incluir información a la ya existente.

Opción	Descripción
r	Lectura (por defecto)
w	Escritura. Trunca el archivo a cero o lo crea si no existe
x	Creación exclusiva. Si el archivo ya existe, el proceso falla
a	Adición. Abre el archivo para agregar datos al final del mismo
b	Modo binario. Agregado a otros modos para leer o escribir archivos binarios. Ej: rb, wb, ab
+	Actualización. Agregado a otros modos para permitir lectura y escritura. Ej: r+

Lectura, escritura
y borrado del
fichero

3



Podemos entender el contenido de un fichero como una secuencia de líneas. Una vez lo abrimos con la función 'open()' podemos iterar sobre el contenido del mismo para aplicar un código.

```
fich_ent = open('nombre_fichero.txt', 'r')
for linea in fich_ent:
    # Procesar la línea
fich_ent.close()
```

El proceso de escritura es igualmente sencillo. Mediante el modo 'w' estamos creando un nuevo fichero o sobrescribiendo si ya existe, y la función 'write()' coge como parámetro el texto que se ha de incluir en el archivo.

```
s = 'Test 1 : Write a file'

with open('filename.txt','w') as f:
    f.write(s)
```

```
with open('filename.txt','r') as f:
    for line in f:
        print(line)
```

Test 1 : Write a file



Eliminar un fichero en Python es una tarea sencilla que se puede realizar utilizando el módulo 'os', que proporciona una interfaz para interactuar con el sistema operativo.

Paso 1: Importar el Módulo 'os'

Paso 2: Usar 'os.remove()'

La función `os.remove()` se utiliza para eliminar un fichero específico. Debes proporcionar la ruta del fichero que deseas eliminar.

Si el archivo no existe, el método `remove` generará un error, por lo que es bueno usar `path.exists` para comprobarlo primero.

```
python

import os

# Ruta del fichero a eliminar
ruta_fichero = 'ruta/al/fichero.txt'

# Comprobar si el fichero existe antes de intentar eliminarlo
if os.path.exists(ruta_fichero):
    try:
        os.remove(ruta_fichero)
        print(f"El fichero '{ruta_fichero}' ha sido eliminado.")
    except PermissionError:
        print(f"No tienes permisos para eliminar el fichero '{ruta_fichero}'")
    except Exception as e:
        print(f"Ocurrió un error: {e}")
else:
    print(f"El fichero '{ruta_fichero}' no existe.")
```




Método	Descripción
seek()	Mueve el puntero hacia el byte indicado
read()	Lee todo el contenido del fichero. Si recibe un número de bytes solo lee los indicados
readline()	Lee una línea del archivo
readlines()	Devuelve una lista en la que cada elemento es una línea del archivo
tell()	Devuelve la posición del puntero
write()	Escribe en el fichero la cadena que reciba
close()	Cierra el archivo



1. Crear y escribir en un fichero:

```
python

with open('ejemplo.txt', 'w') as archivo:
    archivo.write("Primera línea.\nSegunda línea.\nTercera línea.")
```

1. Leer el contenido del archivo:

```
python

with open('ejemplo.txt', 'r') as archivo:
    contenido_completo = archivo.read()
    print("Contenido completo del archivo:")
    print(contenido_completo)
```

1. Volver al inicio del archivo:

```
python

archivo.seek(0)
```

1. Leer una línea:

```
python

primera_linea = archivo.readline()
print("Primera línea del archivo:")
print(primera_linea.strip())
```



1. Leer todas las línea restantes:

```
python

lineas_restantes = archivo.readlines()
print("Líneas restantes del archivo:")
print(lineas_restantes)
```

1. Obtener la posición actual:

```
python

posicion_actual = archivo.tell()
print("Posición actual en el archivo:")
print(posicion_actual)
```

1. Añadir más contenido al archivo:

```
python

with open('ejemplo.txt', 'a') as archivo:
    archivo.write("\nCuarta línea.")
```

1. Verificar el contenido después de añadir más texto:

```
python

with open('ejemplo.txt', 'r') as archivo:
    print("Contenido del archivo después de añadir más texto:")
    print(archivo.read())
```

Gestión del contexto

4



En algunas situaciones, resulta más conveniente automatizar la apertura de un archivo en el modo deseado, es decir, que se cierre automáticamente al concluir las operaciones con él. En Python, esta tarea se lleva a cabo mediante el uso del gestor de contexto.

La declaración `with` en Python se utiliza para asegurar que un recurso sea debidamente manejado, es decir, para gestionar la apertura y cierre de un recurso, como un archivo. Esta declaración simplifica la gestión de recursos y garantiza que ciertos procedimientos (como cerrar un archivo) se realicen de manera segura y automática al salir del bloque `with`.

```
with open('Archivo.txt') as archivo:
    <línea de código>
    ...
    ...
    ...
print('Al salir de la indentación ya estaría cerrado el fichero.')
```

La declaración `with` se utiliza comúnmente con varios tipos de recursos, no solo archivos. Puede aplicarse a objetos que implementan los métodos `__enter__` y `__exit__`, como ciertos contextos de base de datos, conexiones de red, o cualquier otro recurso que requiera una gestión cuidadosa.

Ficheros JSON

5



JSON (JavaScript Object Notation) representa un formato de texto simple diseñado para facilitar el intercambio de datos que en los últimos años ha ganado considerable popularidad. En Python, el formato JSON se puede procesar mediante el módulo `json`. Al ser un formato basado en texto, se vuelve legible tanto para seres humanos como para máquinas. Dentro de los archivos JSON, los datos se organizan en pares de propiedad y valor, adoptando la forma de diccionarios. Estos valores se separan por comas y se encapsulan entre llaves. Además, la propiedad puede contener otro objeto JSON, proporcionando así una notable flexibilidad.

```
import json

with open('data.json') as file:
    data = json.load(file) #Devuelve un diccionario sobre el que se puede iterar.
    #print(data)
print(file.closed)
for client in data['clients']:
    print('Nombre: ', client['first_name'])
    print('Apellido: ', client['last_name'])
    print('Edad: ', client['age'])
    print('Cantidad: ', client['amount'])
    print(' ')
```

```
employees.json

[
  {
    "employee": {
      "firstName": "Lokesh",
      "lastName": "Gupta",
      "website": "howtodoinjava.com"
    }
  },
  {
    "employee": {
      "firstName": "Brian",
      "lastName": "Schultz",
      "website": "example.com"
    }
  }
]
```



Paso de un fichero JSON a un diccionario:
Partimos de un fichero JSON llamado datos.json y lo guardamos en la variable 'datos'

```
python

import json

# Leer el archivo JSON y convertirlo a un diccionario
with open('datos.json', 'r') as archivo_json:
    datos = json.load(archivo_json)

print(datos)

# Salida: {'nombre': 'Juan', 'edad': 30, 'ciudad': 'Madrid'}
```

Paso de un diccionario a un fichero JSON:
Partimos de un diccionario con nombre 'datos' y lo convertimos a un fichero JSON con nombre 'nuevo_datos.json'

```
python

import json

# Convertir el diccionario a un archivo JSON
with open('nuevo_datos.json', 'w') as archivo_json:
    json.dump(datos, archivo_json, indent=4)

print("Archivo JSON creado con éxito.")
```


Ficheros CSV

6



CSV significa valores separados por comas (comma separated values). CSV es un formato de archivo simple que se utiliza para almacenar datos tabulares, como una hoja de cálculo o una base de datos. CSV es un formato de datos muy común en la ciencia de datos.

En el siguiente ejemplo vamos a ver cómo leemos un archivo CSV y lo convertimos en una lista de diccionarios.

```
python

import csv

# Leer el archivo CSV y convertirlo a una lista de diccionarios
with open('datos.csv', mode='r') as archivo_csv:
    lector_csv = csv.DictReader(archivo_csv)
    lista_diccionarios = [fila for fila in lector_csv]

print(lista_diccionarios)
```



Ahora vamos a realizar la operación inversa que es pasar una lista de diccionarios a un archivo CSV.

En la variable 'encabezados' guardamos la claves del diccionario.

Con 'open' abrimos o creamos el fichero en modo escritura.

Con 'csv.DictWriter' creamos un objeto con el que podemos escribir los diccionarios.

Con 'escritor_csv.writeheader' escribimos los encabezados en el CSV.

Con 'escritor_csv.writerows' escribimos las filas en el CSV.

```
python

import csv

# Obtener los encabezados (keys del diccionario)
encabezados = lista_diccionarios[0].keys()

# Escribir la lista de diccionarios en un archivo CSV
with open('nuevo_datos.csv', mode='w', newline='') as archivo_csv:
    escritor_csv = csv.DictWriter(archivo_csv, fieldnames=encabezados)
    escritor_csv.writeheader()
    escritor_csv.writerows(lista_diccionarios)

print("Archivo CSV creado con éxito.")
```

Ficheros XLSX

7



Python proporciona bibliotecas como **‘openpyxl’** para trabajar con archivos XLSX. A continuación se muestra cómo leer un archivo XLSX y convertirlo a una lista de diccionarios y cómo escribir una lista de diccionarios en un archivo XLSX. En este ejemplo veremos se utiliza una biblioteca externa, que se ha instalado con ‘pip’ que se verá en profundidad más adelante.

Primero, creamos un archivo XLSX de ejemplo llamado datos.xlsx con el siguiente contenido:

```
python

from openpyxl import Workbook

# Crear un nuevo libro de trabajo y una hoja
libro = Workbook()
hoja = libro.active

# Añadir encabezados y datos
encabezados = ["nombre", "edad", "ciudad"]
datos = [
    ["Juan", 30, "Madrid"],
    ["Ana", 25, "Barcelona"],
    ["Luis", 35, "Valencia"]
]

hoja.append(encabezados)
for fila in datos:
    hoja.append(fila)

# Guardar el archivo
libro.save("datos.xlsx")
```



Creamos un archivo XLSX partiendo de una lista de diccionarios.

```
python

from openpyxl import Workbook

# Crear un nuevo libro de trabajo y una hoja
libro = Workbook()
hoja = libro.active

# Añadir encabezados y datos
encabezados = ["nombre", "edad", "ciudad"]
datos = [
    ["Juan", 30, "Madrid"],
    ["Ana", 25, "Barcelona"],
    ["Luis", 35, "Valencia"]
]

hoja.append(encabezados)
for fila in datos:
    hoja.append(fila)

# Guardar el archivo
libro.save("datos.xlsx")
```

Creamos una lista de diccionarios partiendo de un archivo XLSX.

```
python

from openpyxl import load_workbook

# Cargar el archivo XLSX
libro = load_workbook("datos.xlsx")
hoja = libro.active

# Leer los datos y convertirlos a una lista de diccionarios
lista_diccionarios = []
encabezados = [celda.value for celda in hoja[1]]

for fila in hoja.iter_rows(min_row=2, values_only=True):
    lista_diccionarios.append(dict(zip(encabezados, fila)))

print(lista_diccionarios)
```

Ficheros XML

8



XML (eXtensible Markup Language) es un lenguaje de marcas diseñado para almacenar y transportar datos. XML se enfoca en describir los datos de una manera que sea tanto legible por humanos como por máquinas. Es extensible, lo que significa que no tiene un conjunto de etiquetas predefinido, y los usuarios pueden definir sus propias etiquetas.

Primero, creamos un archivo XML de ejemplo llamado `personas.xml` con el siguiente contenido:

```
python

import xml.etree.ElementTree as ET

# Lista de diccionarios, donde cada diccionario es una persona
personas_list = [
    {'nombre': 'Juan', 'edad': '30'},
    {'nombre': 'María', 'edad': '25'}
]

# Crear el elemento raíz
root = ET.Element("personas")

# Iterar sobre la lista de diccionarios y crear el XML
for persona in personas_list:
    persona_element = ET.SubElement(root, "persona")
    ET.SubElement(persona_element, "nombre").text = persona['nombre']
    ET.SubElement(persona_element, "edad").text = persona['edad']

# Crear el árbol XML y escribirlo a un archivo llamado 'personas.xml'
tree = ET.ElementTree(root)
tree.write("personas.xml", encoding="utf-8", xml_declaration=True)

print("Archivo personas.xml creado correctamente.")
```




- Para cada persona encontrada en el XML, creamos un diccionario que contiene el nombre y la edad.
- En cada iteración, añadimos el diccionario persona_obj a la lista personas_list.
- Al final del código, se imprime toda la lista de diccionarios, que representa a todas las personas extraídas del XML.

Cargar un fichero XML e imprimirlo por pantalla

```
python

import xml.etree.ElementTree as ET

# Abrir el archivo usando la función open
with open('personas.xml', 'r', encoding='utf-8') as file:
    xml_content = file.read()

# Analizar el contenido XML
root = ET.fromstring(xml_content)

# Lista para almacenar los datos de las personas
personas_list = []

# Iterar sobre las personas y extraer los datos
for persona in root.findall('persona'):
    nombre = persona.find('nombre').text
    edad = persona.find('edad').text

    # Crear un diccionario para cada persona
    persona_obj = {
        'nombre': nombre,
        'edad': edad
    }

    # Añadir el diccionario al array (lista)
    personas_list.append(persona_obj)

# Imprimir el array completo
print(personas_list)

#[{'nombre': 'Juan', 'edad': '30'}, {'nombre': 'María', 'edad': '25'}]
```

¡Gracias!